

Mini-projet 19 — Une CLI propre (argparse)

Pourquoi ce mini-projet ?

Ton scanner marche, mais il faut éditer le code pour changer la cible. Un vrai outil se pilote **depuis le terminal** : `python toolkit.py 127.0.0.1 --ports 1-1000 --timeout 0.5`. Le module `argparse` transforme ton script en **commande** : il lit les arguments, les vérifie, génère l'aide automatiquement. C'est ce qui rend ton projet utilisable par quelqu'un d'autre que toi — et ça fait très « pro » en entretien.

1. Le problème avec sys.argv « à la main »

Python te donne les arguments bruts dans `sys.argv` (une liste de chaînes). Mais tout gérer à la main est pénible :

```
import sys
print(sys.argv)  # ['toolkit.py', '127.0.0.1', '--ports', '1-1000'] — à toi de tout démêler
```

Il faudrait vérifier le nombre d'arguments, convertir les types, gérer les options manquantes, écrire l'aide... `argparse` fait **tout ça** pour toi.

2. ArgumentParser + add_argument

```
import argparse

parser = argparse.ArgumentParser(description="Scanner de ports")
parser.add_argument("cible", help="l'IP à scanner")          # positionnel (obligatoire)
parser.add_argument("--ports", default="1-100", help="plage, ex: 1-1000") # optionnel

args = parser.parse_args()  # lit la ligne de commande et remplit "args"
print(args.cible)          # l'IP donnée
print(args.ports)          # la plage (ou "1-100" par défaut)
```

Après `parse_args()`, tout est dans `args` : chaque argument devient un attribut (`args.cible`, `args.ports`). Le nom de l'attribut = le nom de l'argument, sans les `--`.

3. Positionnel vs optionnel

Type	Déclaration	À l'usage
Positionnel (obligatoire)	<code>add_argument("cible")</code>	<code>toolkit.py 127.0.0.1</code> — juste la valeur, par position
Optionnel (facultatif)	<code>add_argument("--ports")</code>	<code>toolkit.py 127.0.0.1 --ports 1-500</code> — nommé avec <code>--</code>

C'est le **même « positionnel vs nommé » que le MP02** (les `*args` / `**kwargs`), mais côté ligne de commande. Un positionnel est requis ; un `--optionnel` a souvent une valeur par défaut.

4. Contrôler les valeurs : type, action, choices

```
parser.add_argument("--timeout", type=float, default=1.0)
# type=float -> argparse convertit "0.5" (texte) en 0.5 (float) TOUT SEUL

parser.add_argument("--verbose", action="store_true")
# un DRAPEAU : présent -> True, absent -> False (pas de valeur à donner)

parser.add_argument("--plugin", default="scan", choices=["ping", "scan", "all"])
# choices -> argparse REFUSE toute valeur hors de la liste, automatiquement
```

- `type=float` / `type=int` → conversion automatique (fini le `int(...)` à la main du MP01 !).
- `action="store_true"` → un drapeau on/off : `--verbose` présent = `True`.
- `choices=[...]` → validation automatique : une valeur hors liste est rejetée avec un message clair.

5. Cadeau gratuit : l'aide et les erreurs

argparse génère **tout seul** le message d'aide (`-h / --help`) et gère les erreurs :

```
$ python toolkit.py --help
usage: toolkit.py [-h] [--ports PORTS] [--timeout TIMEOUT] [--verbose] cible

Scanner de ports

positional arguments:
  cible                l'IP à scanner
...

$ python toolkit.py # sans cible
usage: toolkit.py [-h] ... cible
toolkit.py: error: the following arguments are required: cible
```

Tu n'écris **rien** pour ça : le `help=` de chaque argument alimente l'aide, et les arguments manquants ou invalides déclenchent un message d'erreur propre. C'est ça, une CLI « pro » offerte.

🎯 Ta mission

Objectif : une CLI `toolkit.py` qui parse les arguments d'un scan.

- Une petite fonction `parse_plage(texte)` qui transforme `"1-1000"` en deux entiers `(1, 1000)` (un `split("-")` + `int`, tes MP01).
- Un `ArgumentParser` avec :
 - `cible` → positionnel (l'IP) ;
 - `--ports` → défaut `"1-100"` ;
 - `--timeout` → `type=float`, défaut `1.0` ;
 - `--verbose` → `action="store_true"`.
- Après `parse_args()`, affiche un récap propre du scan demandé.

Tests (dans le terminal) :

```
$ python toolkit.py 127.0.0.1
cible=127.0.0.1 ports=1-100 timeout=1.0 verbose=False

$ python toolkit.py 127.0.0.1 --ports 1-1000 --timeout 0.5 --verbose
cible=127.0.0.1 ports=1-1000 timeout=0.5 verbose=True

$ python toolkit.py # sans cible -> argparse affiche une erreur claire (voulu !)
$ python toolkit.py --help # -> l'aide générée automatiquement
```

♣ **Bonus (l'assemblage réel) :** branche ta CLI sur ton scanner ! Importe `scan_parallele` (MP16) ou `scan_async` (MP17), et après le parsing, lance un vrai scan sur `args.cible` et la plage, puis affiche les ports ouverts. Là, `python toolkit.py 127.0.0.1 --ports 8000-8100` devient un **vrai outil**.

✓ Critères de réussite

- `python toolkit.py 127.0.0.1` marche avec les valeurs par défaut.
- `--timeout 0.5` arrive bien en `float` (pas en chaîne) grâce à `type=float`.
- `--verbose` vaut `True` quand présent, `False` sinon.
- Sans `cible`, argparse affiche une erreur (tu n'écris pas cette gestion toi-même).

Si tu bloques — indices progressifs

Étape	Indice
Le parser	<code>parser = argparse.ArgumentParser(description="...")</code> puis les <code>add_argument</code> .
Positionnel	<code>parser.add_argument("cible")</code> — sans <code>--</code> .
Optionnels	<code>parser.add_argument("--ports", default="1-100")</code> , idem timeout (<code>type=float</code>) et verbose (<code>action="store_true"</code>).

Récupérer	<code>args = parser.parse_args()</code> puis <code>args.cible</code> , <code>args.ports</code> , etc.
parse_plage	<code>debut, fin = texte.split("-")</code> puis <code>return int(debut), int(fin)</code> .
Structure	Mets tout dans une fonction <code>main()</code> et appelle-la sous <code>if __name__ == "__main__":</code> .

Où ranger : `mini-projets/19-cli-argparse/toolkit.py` .

Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé » + colle-moi la sortie de tes 2 premiers tests. Après, le MP20 assemble TOUT en un seul outil pro.